Engineering Principles & Practices

IITM Explainable Slotbook | Nishanth Senthil Kumar (EE23B049) & Sparsh Gupta (EE23B117)

Project lifecycle

Our project converts IITM-style course, slot, conflict, instructor-preference, and synthetic student-demand data into an optimized slotbook. The lifecycle is deliberately traceable: reviewed prompt -> small implementation change -> automated verification -> documentation update -> Git commit. The solver produces structured evidence, and explanations are derived from that evidence rather than generated independently.

1. Prompt engineering

We treat prompts as versioned engineering inputs. Every substantial task is framed with **CoSTAR**: Context identifies the current architecture and data assumptions; Objective states one testable outcome; Style constrains implementation structure; Tone controls the level of certainty; Audience names the evaluator, stakeholder, or maintainer; Response specifies concrete deliverables. Each prompt also contains acceptance criteria and explicit negative instructions.

Review gate	Question asked before accepting output
Specificity	Are files, interfaces, inputs, outputs, and scope named?
Assumptions	Are unknown institutional facts labelled or requested?
Safety/privacy	Does the output exclude identities, secrets, and fabricated data?
Architecture	Does the change respect parser/validator/solver/UI boundaries?
Verification	Is there a test or observable acceptance command?

Representative negative instructions include: do not represent synthetic preferences as real IITM data; do not let an LLM decide or justify slots; do not ignore unknown identifiers; do not place optimization logic in JavaFX controllers; and do not claim global optimality unless CP-SAT returns OPTIMAL.

2. Structured debugging

We use an eight-step cycle: **reproduce, observe, hypothesize, isolate, correct, test, regress, document**. Failures are first reduced to one command or minimal CSV fixture. We collect the input row, exception or solver status, expected result, and relevant module. Competing hypotheses are listed before code is edited. The smallest correction is made at the module that owns the behaviour, a regression assertion is added, and then the complete suite is rerun.

Observed	Exporter test failed although both CSV files existed.
Hypotheses	Missing output; empty explanations; expectation aimed at the wrong formatter.
Evidence	CSV contained course IDs and alternative descriptions, but not the UI heading.
Root cause	Test expected text-view presentation inside structured CSV output.
Correction	Assert the semantic marker 'rejected because'.
Regression	All 11 tests rerun and passed.

3. Version control

Commits are coherent, reviewable increments rather than end-of-project snapshots. The initial sequence separates architecture and prompt governance, mock data, the solver/application, and evaluator documentation. Before committing we inspect the diff and run the relevant tests. Generated outputs, build directories, IDE settings, and secrets are ignored. Prompt files live under `prompts/` and are committed with the code they influenced, preserving revisions, rejected assumptions, debugging evidence, and verification results.

Increment	Evidence committed
Architecture	Maven skeleton, module flow, prompt checklist
Data	Feasible and infeasible fixtures, 30 synthetic preference profiles
Application	Parser, validator, CP-SAT model, explanations, CLI, UI, tests
Documentation	Manual, code walkthrough, test plan, debugging guide, presentation

4. Testing, refactoring, and documentation

Tests target guarantees rather than implementation details. They cover quoted CSV parsing, fixture loading, student co-demand aggregation, valid and contradictory inputs, complete assignment, allowed and fixed slots, every hard-conflict pair, explanation completeness, infeasibility, and both exports. Because several timetables may be equally optimal, tests assert invariants instead of one fragile assignment. The JavaFX interface has a separate manual smoke checklist for the headless test environment.

Refactoring is supported by one-way dependencies: immutable model records; CSV translation; deterministic validation; CP-SAT formulation; evidence-based explanation; an application service; and thin CLI/UI adapters. This means a live change such as a new validation rule can be implemented in one class with one focused test. Public concepts have JavaDoc. The repository also includes a man-page-style manual with NAME, SYNOPSIS, DESCRIPTION, OPTIONS, INPUTS, OUTPUTS, EXAMPLES, EXIT STATUS, and LIMITATIONS; plus architecture, mathematical walkthrough, test plan, debugging guide, and a two-minute script.

Appendix A - sample reviewed prompts

Architecture prompt. Context: Java 17 IITM slotting application for an individual viva. **Objective:** define modular boundaries for parsing, validation, optimization, explanation, UI, export, and testing. **Response:** repository structure and verification plan before implementation. **Negative constraints:** do not hide solver logic in controllers; do not claim mock inputs are real; do not use an LLM to invent reasons.

Testing prompt. Context: parser, validator, CP-SAT solver, explanation builder, exporter, CLI, and UI exist. **Objective:** test observable guarantees and important failure paths. **Response:** JUnit tests for parsing, validation, completeness, domains, fixed slots, conflicts, explanations, infeasibility, and exports. **Negative constraints:** do not assert one exact optimum; do not test private variable names; do not weaken validation to make tests pass.

The complete CoSTAR prompts, prompt reviews, acceptance decisions, negative prompts, and debugging records are versioned in `prompts/PROMPT_LOG.md`. This keeps the three-page submission concise while preserving a full auditable history in Git.